
Workforce Planning & Copilot Execution Pack

Reference guide for the packaged Markdown files, installation workflow, and correct file usage.

What this package gives you

A complete planning and execution chain for Workforce: module specs, master-plan formatting rules, atomic execution prompts, decision rules, progress templates, status vocabulary, a full Workforce Studio example set, and a Copilot-ready install prompt.

Best use: define a module -> convert it into a plan -> execute one checkbox at a time -> log progress consistently -> keep tenancy, RBAC, and scope discipline intact.

Prepared package contents include individual Markdown files with clear filenames, a packaged PDF reference guide, and a Copilot-install prompt set.

1. How the framework works

This package creates a repeatable workflow for building Workforce modules without losing structure. The main chain is: **module spec -> master-plan section -> atomic execution plan -> Copilot execution prompt -> decision rules -> progress report -> status vocabulary**.

Recommended usage order

- 1 Start with **MODULE_SPEC_TEMPLATE.md** to define the module cleanly.
- 2 Use **MASTER_PLAN_FORMAT_STANDARD.md** to convert the module into a consistent plan section.
- 3 Use **PLAN_STATUS_VOCABULARY.md** and **RUN_PLAN_DECISION_RULES.md** to standardize executor behavior.
- 4 Use **COPILOT_EXECUTE_ATOMIC_TASK.md** and **COPILOT_EXECUTION_WRAPPERS.md** to drive one-checkbox implementation.
- 5 Use a progress template to record what happened and why.
- 6 Use the Workforce Studio example files as a concrete model.
- 7 Use the Copilot install files when you want the planning system itself installed into a repository.

Core safety rules

- Never weaken tenant isolation.
- Never bypass RBAC.
- Never let Workforce Studio mutate live operational Workforce data in v1.
- Prefer additive, reviewable patches.
- Keep one logical patch target per checkbox whenever possible.

2. File catalog

The table below lists every packaged file, its purpose, and the correct time to use it.

Filename	Purpose	Use when
00_README_FIRST.md	Package overview, file map, usage order, and safety reminders.	Open this first to understand the bundle and the recommended reading order.
templates/MODULE_SPEC_TEMPLATE.md	Structured design template for new modules.	Use before planning. This is the design-source document.
templates/MASTER_PLAN_FORMAT_STANDARD.md	Rules for writing clean, executor-friendly master-plan sections.	Use when converting a module spec into a plan section.
templates/PLAN_STATUS_VOCABULARY.md	Normalized plan states and executor outcome codes.	Use in plans, progress files, and executor logs.

Filename	Purpose	Use when
templates/RUN_PLAN_DECISION_RULES.md	Skip/block/retry/stop policy for atomic execution.	Use during executor runs to prevent loops and scope drift.
templates/COPILOT_EXECUTE_ATOMIC_TASK.md	Hard rules for exactly-one-checkbox Copilot execution.	Use as the primary execution instruction file.
templates/COPILOT_EXECUTION_WRAP_PERS.md	Copyable prompts for task execution and progress updates.	Use when sending a targeted prompt to Copilot.
templates/PROGRESS_REPORT_TEMPLATE.md	Full progress-report template with task log and blocker sections.	Use when you want detailed run tracking.
templates/PROGRESS_REPORT_TEMPLATE_SHORT.md	Shorter executor-oriented progress template.	Use when you want light-weight status tracking.
workforce_studio/WORKFORCE_STUDIO_IMPLEMENTATION_SPEC.md	Detailed Studio module spec covering scope, data model, API, services, validation, and rollout.	Use as a concrete reference for how a real module spec should look.
workforce_studio/WORKFORCE_STUDIO_COPILOT_HANDOFF.md	Copilot-oriented implementation brief for Studio.	Use when handing the module to Copilot for phased execution.
workforce_studio/HN3T_MASTER_PLAN_WORKFORCE_STUDIO_BUILD_ORDER.md	Phased build-order plan section for Studio.	Use for high-level roadmap planning.
workforce_studio/HN3T_MASTER_PLAN_WORKFORCE_STUDIO_ATOMIC_EXECUTION_PLAN.md	Strict atomic checkbox plan for Studio.	Use with `run_plan.sh` or exact-checkbox execution.
copilot_install/COPILOT_INSTALL_WORKFORCE_PLANNING_SETUP.md	Instructions for installing this framework into a repo.	Use when the goal is to bootstrap the planning system itself.
copilot_install/COPILOT_INSTALL_WORKFORCE_PLANNING_SETUP_PROMPT.md	Ready-to-paste install prompt for Copilot.	Use when you want Copilot to perform the installation work.
manifest.txt	Machine-readable list of included files.	Use for packaging, verification, or scriptable file listing.

3. Correct usage by file group

Templates

MODULE_SPEC_TEMPLATE.md — Use this before planning. It is where you define module identity, scope, ownership boundaries, RBAC, domain model, API draft, UI surfaces, rollout plan, and acceptance criteria.

MASTER_PLAN_FORMAT_STANDARD.md — Use this when translating a spec into a real plan section. It keeps headings, checkbox phrasing, endpoint notation, and task granularity consistent.

PLAN_STATUS_VOCABULARY.md — Use this to prevent fuzzy status wording. It keeps plan state and executor outcomes aligned.

RUN_PLAN_DECISION_RULES.md — Use this when the executor needs to know whether to run, skip, block, retry, or stop on a checkbox.

COPILOT_EXECUTE_ATOMIC_TASK.md — Use this as the hard contract whenever you want Copilot to emit one narrow unified diff patch.

COPILOT_EXECUTION_WRAPPERS.md — Use these prompts when you want ready-made commands for next-task execution, exact-checkbox execution, progress updates, or decision-rule enforcement.

PROGRESS_REPORT templates — Use the full version when you want long-form evidence and blocker logging. Use the short version when the repo favors minimal executor reporting.

Workforce Studio example set

WORKFORCE_STUDIO_IMPLEMENTATION_SPEC.md — Use this as a model of what a serious Workforce module spec should contain before any build execution starts.

WORKFORCE_STUDIO_COPILOT_HANDOFF.md — Use this as the bridge between product design and Copilot implementation. It reframes the spec into actionable build instructions.

Build-order plan section — Use this when you want a phased roadmap that groups work into larger project milestones.

Atomic execution plan — Use this when the repo is ready for one-checkbox-at-a-time implementation with ``run_plan.sh`` or a similar loop.

Copilot install set

COPILOT_INSTALL_WORKFORCE_PLANNING_SETUP.md — Use this to tell Copilot how to install the planning system itself into the repo, where to place the files, and what repo guidance to update.

COPILOT_INSTALL_WORKFORCE_PLANNING_SETUP_PROMPT.md — Use this when you want a single ready-to-paste prompt that tells Copilot to perform the install task and nothing else.

4. Recommended installation into a repo

A good default target is a dedicated planning docs subtree. This keeps the framework easy to discover without forcing it into the application code tree.

Suggested layout

```
docs/  
  planning/  
    README.md  
    templates/
```

```
workforce_studio/
```

- 1 Inspect existing repo conventions first. Do not impose a new docs layout if the repo already has a strong one.
- 2 Install the templates and example files as additive documentation.
- 3 Update `copilot-instructions.md`, `AGENTS.md`, or equivalent so future Copilot sessions know these planning docs exist.
- 4 Link or append the Workforce Studio master-plan section carefully instead of overwriting existing plan content.
- 5 Create or merge `PROGRESS\_REPORT.md` without deleting useful history.
- 6 Keep the installation task separate from feature implementation.

Copilot install prompt intent

The packaged install prompt is meant to bootstrap the planning system only. It should not begin implementing Workforce Studio itself. That separation is important because installing planning infrastructure is a documentation and process task, not a feature delivery task.

5. Practical operating flow

Step 1 — Define the module

Start with ``MODULE_SPEC_TEMPLATE.md`` and write the real source design. Clarify what the module owns, what it only references, what it must not mutate, and which RBAC/tenant rules apply.

Step 2 — Convert the design into a plan

Use ``MASTER_PLAN_FORMAT_STANDARD.md`` to turn the spec into a plan section with dependency-ordered phases and atomic checkboxes.

Step 3 — Standardize status handling

Use ``PLAN_STATUS_VOCABULARY.md`` so complete, blocked, skipped, superseded, retry, and stop states always mean the same thing.

Step 4 — Enforce executor discipline

Use ``RUN_PLAN_DECISION_RULES.md`` to avoid noisy loops, bad retries, and accidental scope creep.

Step 5 — Execute one checkbox at a time

Use ``COPILOT_EXECUTE_ATOMIC_TASK.md`` plus an execution wrapper prompt. The ideal output is one small unified diff patch.

Step 6 — Record what happened

Use one of the progress-report templates so the run is inspectable and recoverable.

Step 7 — Reuse the Studio example

Compare the new module against the Workforce Studio files if you need a real example of the expected shape.

Common mistakes this framework is meant to prevent

- Starting implementation before module ownership boundaries are defined
- Bundling migrations, models, services, routes, tests, and frontend into a single checkbox
- Allowing Copilot to drift into future phases
- Using vague statuses like 'mostly done' or 'kind of blocked'
- Letting install/bootstrap tasks mutate application logic unnecessarily

6. Why Workforce Studio is included

The Workforce Studio example set is included so the framework is not purely abstract. It shows how the templates and standards look when applied to a real Workforce module: scope-safe, RBAC-aware, staged, and Copilot-ready.

Studio-specific hard rule

For Workforce Studio v1, any patch that mutates live operational Workforce data, bypasses RBAC, bypasses tenant filtering, or bundles future Studio phases into the current checkbox should be rejected as a safety or scope problem.

Appendix A — Recommended Copilot install prompt

Install the Workforce planning and Copilot execution framework from the provided package into this repository.

Primary reference file:

- COPILOT_INSTALL_WORKFORCE_PLANNING_SETUP.md

Installation requirements:

- inspect existing repo conventions first
- install these files into a sensible docs/planning structure, or the nearest equivalent
- create docs/planning/README.md from the package overview
- update Copilot guidance files so future sessions know these planning docs exist
- do not overwrite existing master-plan content unnecessarily
- preserve tenant isolation, RBAC assumptions, and design-only Studio boundaries
- keep the patch additive and reviewable
- output only a unified diff patch beginning with diff --git

Do not implement Workforce Studio itself in this task.

This task is only for installing the planning framework and its documentation into the repository.

Appendix B — Quick package map

Root: package overview and manifest. Templates: reusable planning and execution files. Workforce Studio: concrete example documents. Copilot install: repo bootstrap instructions and ready-made install prompt. Reference: this PDF and the Markdown source guide.